

# AAD Dynamic Programming Solved Problems

## 1 Cloth Cutting

We are given a rectangular piece of cloth of size  $X \times Y$  and  $n$  products that can be made from the cloth. Product  $i$  requires a rectangular piece of size  $a_i \times b_i$  and can be sold for  $c_i$ . The cloth can be repeatedly cut either horizontally or vertically. After a cut, the resulting rectangular pieces can be cut further independently. We may make any number of copies of each product, and some cloth may be left unused. The goal is to determine the maximum possible selling price. We assume that a product can be rotated, so a product requiring  $a_i \times b_i$  can also be made from a  $b_i \times a_i$  piece.

### 1.1 Dynamic Programming Formulation

The important observation is that after making the first cut, the two resulting pieces are independent. Thus, we can solve the problem for each smaller rectangle and combine their optimal values. Define

$$DP[x][y] = \text{maximum revenue obtainable from an } x \times y \text{ rectangle.}$$

Also define

$$P[x][y] = \text{maximum price of a product requiring exactly an } x \times y \text{ piece.}$$

If no product has these dimensions, set  $P[x][y] = 0$ . For every rectangle  $x \times y$ , there are three possibilities.

#### Case 1: Do not cut the rectangle

If a product of size  $x \times y$  exists, we can sell it directly:

$$DP[x][y] \geq P[x][y].$$

#### Case 2: Make a vertical cut

If we cut after  $k$  units, where  $1 \leq k < x$ , the rectangle becomes

$$k \times y \quad \text{and} \quad (x - k) \times y.$$

The best revenue from this cut is therefore

$$DP[k][y] + DP[x - k][y].$$

#### Case 3: Make a horizontal cut

Similarly, a horizontal cut after  $k$  units produces

$$x \times k \quad \text{and} \quad x \times (y - k),$$

giving revenue

$$DP[x][k] + DP[x][y - k].$$

Therefore the recurrence is

$$DP[x][y] = \max \left\{ P[x][y], \max_{1 \leq k < x} \{ DP[k][y] + DP[x - k][y] \}, \max_{1 \leq k < y} \{ DP[x][k] + DP[x][y - k] \} \right\}.$$

We compute the table in increasing order of  $x$  and  $y$ , since every state on the right hand side has at least one smaller dimension.

## 1.2 Algorithm

Before computing the DP table, store the best price for every possible rectangle size. For every product  $(a_i, b_i, c_i)$ , store  $c_i$  at both  $(a_i, b_i)$  and  $(b_i, a_i)$  when rotation is allowed.

---

### Algorithm 1 Maximum Revenue

---

```

1: for  $x = 0, \dots, X$  do
2:   for  $y = 0, \dots, Y$  do
3:      $P[x][y] \leftarrow 0$ 
4:   end for
5: end for
6: for each product  $(a, b, c)$  do
7:    $P[a][b] \leftarrow \max(P[a][b], c)$ 
8:    $P[b][a] \leftarrow \max(P[b][a], c)$ 
9: end for
10: for  $x = 1, \dots, X$  do
11:   for  $y = 1, \dots, Y$  do
12:      $DP[x][y] \leftarrow P[x][y]$ 
13:     for  $k = 1, \dots, x - 1$  do
14:        $DP[x][y] \leftarrow \max(DP[x][y], DP[k][y] + DP[x - k][y])$ 
15:     end for
16:     for  $k = 1, \dots, y - 1$  do
17:        $DP[x][y] \leftarrow \max(DP[x][y], DP[x][k] + DP[x][y - k])$ 
18:     end for
19:   end for
20: end for
21: return  $DP[X][Y]$ 

```

---

## 1.3 Example

Suppose the cloth is  $4 \times 4$  and the available products are

| Product | Size         | Price |
|---------|--------------|-------|
| $A$     | $1 \times 1$ | 2     |
| $B$     | $2 \times 2$ | 7     |
| $C$     | $4 \times 1$ | 5     |
| $D$     | $4 \times 4$ | 20    |

Although product  $D$  gives 20 when the entire cloth is used, we can do better by cutting the cloth into four  $2 \times 2$  pieces:

$$4 \times 4 \longrightarrow 4 \times 2 + 4 \times 2 \longrightarrow 2 \times 2 + 2 \times 2 + 2 \times 2 + 2 \times 2.$$

Each  $2 \times 2$  piece is used for product  $B$ , giving

$$7 + 7 + 7 + 7 = 28.$$

Thus

$$\boxed{DP[4][4] = 28}.$$

## 1.4 Reconstructing the Actual Solution

The DP value tells us the maximum revenue, but not automatically how to achieve it. To reconstruct the solution, maintain another table

$$choice[x][y].$$

Whenever  $DP[x][y]$  is computed, store the decision that produced its value:

- **PRODUCT(i)** if product  $i$  is used directly;
- **VERTICAL(k)** if the optimal solution makes a vertical cut after  $k$  units;
- **HORIZONTAL(k)** if the optimal solution makes a horizontal cut after  $k$  units.

For example, for the example above, the choices could be

$$\begin{aligned} \text{choice}[4][4] &= \text{HORIZONTAL}(2), \\ \text{choice}[4][2] &= \text{VERTICAL}(2), \\ \text{choice}[2][2] &= \text{PRODUCT}(B). \end{aligned}$$

Starting from  $(X, Y)$ , follow these choices recursively:

---

**Algorithm 2** Reconstruct Cutting Plan

---

```

1: procedure RECONSTRUCT( $x, y$ )
2:   if  $\text{choice}[x][y] = \text{PRODUCT}(i)$  then
3:     Output “Make product  $i$  from  $x \times y$  piece”
4:   else if  $\text{choice}[x][y] = \text{VERTICAL}(k)$  then
5:     Output “Vertical cut at  $k$ ”
6:     RECONSTRUCT( $k, y$ )
7:     RECONSTRUCT( $x - k, y$ )
8:   else if  $\text{choice}[x][y] = \text{HORIZONTAL}(k)$  then
9:     Output “Horizontal cut at  $k$ ”
10:    RECONSTRUCT( $x, k$ )
11:    RECONSTRUCT( $x, y - k$ )
12:   end if
13: end procedure

```

---

For the  $4 \times 4$  example, reconstruction produces:

$$\begin{array}{c} 4 \times 4 \\ \downarrow \text{horizontal cut at 2} \\ \begin{array}{cc} 4 \times 2 & 4 \times 2 \end{array} \\ \downarrow \qquad \qquad \downarrow \\ \begin{array}{cc} 2 \times 2 + 2 \times 2 & 2 \times 2 + 2 \times 2 \end{array} \\ \downarrow \qquad \qquad \downarrow \\ B + B + B + B. \end{array}$$

Hence the reconstructed plan makes four copies of product  $B$  and achieves the maximum revenue of 28.

## 1.5 Correctness and Complexity

Consider an optimal solution for an  $x \times y$  rectangle. Its first operation is either (i) using the entire rectangle as one product, or (ii) making a horizontal or vertical cut. In case (i), its value is captured by  $P[x][y]$ . In case (ii), the two resulting rectangles are independent, and by the definition of  $DP$ , their optimal values are exactly the corresponding DP entries in the recurrence. Therefore every possible optimal solution is considered, and taking the maximum gives the optimal value. There are  $XY$  DP states. Each state considers  $O(x + y)$  cuts, giving

$$\boxed{O(XY(X + Y))}$$

time and

$$\boxed{O(XY)}$$

space. Processing the  $n$  products takes an additional  $O(n)$  time.

## 2 Pebbling a Checkerboard

We are given a checkerboard with 4 rows and  $n$  columns. Each square contains an integer. We also have  $2n$  pebbles and may place some or all of them on the board. A placement is *legal* if no two pebbles are horizontally or vertically adjacent; diagonal adjacency is allowed. The goal is to maximize the sum of the integers in the squares covered by pebbles.

We will solve this using dynamic programming. The key observation is that, when processing the board from left to right, the only information about the previous columns that matters is the pattern of pebbles in the immediately preceding column.

## 2.1 Legal patterns in one column

Consider one column. Since there are four rows, a pattern can be represented by a 4-bit string: a 1 means that a pebble is placed in that row and a 0 means that the row is empty.

Since vertically adjacent pebbles are forbidden, a legal pattern cannot contain two consecutive 1's. Thus, among the  $2^4 = 16$  possible bit strings, the legal ones are

$$\boxed{0000, 0001, 0010, 0100, 0101, 1000, 1001, 1010}.$$

Hence there are exactly

$$\boxed{8}$$

possible patterns.

Notice also that every legal pattern contains at most two pebbles. Indeed, among four vertically ordered squares, three pebbles would necessarily put two of them in consecutive rows. Therefore at most  $2n$  pebbles can occur in any legal placement, so the given supply of  $2n$  pebbles never prevents a legal placement.

We number the patterns as follows:

| Type | Pattern | Number of pebbles |
|------|---------|-------------------|
| 0    | 0000    | 0                 |
| 1    | 0001    | 1                 |
| 2    | 0010    | 1                 |
| 3    | 0100    | 1                 |
| 4    | 0101    | 2                 |
| 5    | 1000    | 1                 |
| 6    | 1001    | 2                 |
| 7    | 1010    | 2                 |

## 2.2 Compatibility of patterns

Suppose pattern  $p$  is used in column  $k$  and pattern  $q$  is used in column  $k + 1$ . Both patterns are individually legal, so there are no vertical conflicts. The only possible new conflict is horizontal: two pebbles in the same row of adjacent columns would be forbidden.

Thus  $p$  and  $q$  are *compatible* exactly when they do not have a 1 in the same position. If the patterns are viewed as 4-bit integers, this is equivalent to

$$\boxed{p \& q = 0},$$

where  $\&$  denotes bitwise AND.

For example,

$$0101 \& 1000 = 0000,$$

so 0101 and 1000 are compatible. On the other hand,

$$0101 \& 0100 = 0100 \neq 0000,$$

so 0101 and 0100 are not compatible.

Since there are only eight patterns, all compatibility relationships can be computed once before running the dynamic program.

## 2.3 Dynamic programming

Let  $A[r, k]$  denote the integer written in row  $r$  and column  $k$ . For a pattern  $p$ , define

$$w(k, p) = \sum_{\substack{1 \leq r \leq 4 \\ \text{bit } r \text{ of } p = 1}} A[r, k].$$

Thus  $w(k, p)$  is the contribution obtained by placing pattern  $p$  in column  $k$ .

Define

$$DP[k, p]$$

to be the maximum sum obtainable using the first  $k$  columns, subject to the condition that pattern  $p$  is used in column  $k$ .

For the first column there is no compatibility restriction, so

$$DP[1, p] = w(1, p).$$

Now consider  $k > 1$ . If pattern  $p$  is used in column  $k$ , the pattern  $q$  in column  $k - 1$  must be compatible with  $p$ . Among all such patterns, we choose the one giving the largest value:

$$DP[k, p] = w(k, p) + \max_{\substack{q \text{ is a legal pattern} \\ p \& q = 0}} DP[k - 1, q].$$

After processing all  $n$  columns, the last column can have any of the eight patterns. Therefore the optimal value is

$$\max_p DP[n, p].$$

## 2.4 Algorithm

The resulting algorithm is:

---

### Algorithm 3 Optimal Pebbling

---

```

1:  $P \leftarrow \{0000, 0001, 0010, 0100, 0101, 1000, 1001, 1010\}$ 
2: for all  $p \in P$  do
3:    $DP[1, p] \leftarrow w(1, p)$ 
4: end for
5: for  $k = 2, \dots, n$  do
6:   for all  $p \in P$  do
7:      $DP[k, p] \leftarrow -\infty$ 
8:     for all  $q \in P$  do
9:       if  $p \& q = 0$  then
10:         $DP[k, p] \leftarrow \max(DP[k, p], DP[k - 1, q] + w(k, p))$ 
11:       end if
12:     end for
13:   end for
14: end for
15: return  $\max_{p \in P} DP[n, p]$ 

```

---

## 2.5 Correctness

We prove that the dynamic program computes an optimal legal placement.

Consider any state  $DP[k, p]$ . By definition, column  $k$  has pattern  $p$ . Any legal placement of the first  $k$  columns must have some pattern  $q$  in column  $k - 1$ . Since the two columns are adjacent,  $q$  and  $p$  must be compatible. Therefore every legal placement counted by  $DP[k, p]$  has value at most

$$w(k, p) + \max_{q: p \& q = 0} DP[k - 1, q].$$

Conversely, choose a pattern  $q$  attaining the maximum in the recurrence. By the induction hypothesis,  $DP[k - 1, q]$  corresponds to a legal placement of the first  $k - 1$  columns. Since  $q$  and  $p$  are compatible, adding pattern  $p$  in column  $k$  creates no horizontal conflict, and  $p$  itself creates no vertical conflict. Hence the result is a legal placement of the first  $k$  columns with exactly the value specified by the recurrence.

Thus, by induction on  $k$ , every  $DP[k, p]$  is optimal for its stated condition. Taking the maximum over the eight possible final patterns therefore gives a globally optimal placement.

## 2.6 Running time and reconstruction

There are only 8 states for each column. For every current pattern we examine at most 8 previous patterns. Thus each column requires at most

$$8 \cdot 8 = 64$$

constant-time operations. Consequently,

$$\boxed{\text{Running time} = O(n)}.$$

If only the maximum value is required, we need to store only the previous and current eight DP values, giving  $O(1)$  additional space.

To reconstruct the actual placement, store

$$parent[k, p]$$

equal to the pattern  $q$  that maximizes the recurrence for  $DP[k, p]$ . After finding the pattern  $p^*$  maximizing  $DP[n, p]$ , trace backwards:

$$p_n = p^*, \quad p_{k-1} = parent[k, p_k].$$

This gives the pattern used in every column, and hence the locations of all pebbles.

$\boxed{\text{The essential DP idea: remember only the pattern in the last column.}}$